VERITAS.

Evidence-Driven Generative AI for Reliable Research

Complete Project Documentation

From the basics: what we built, every concept we used, how every number is calculated,
and a slide-by-slide explanation of the Review 2 presentation.

21CSE306P · Applied Generative AI · B.Tech 5th Semester (2026–27)

Om Pathania (RA2411003012495)

Faculty: Dr. Vaishnavi Moorthy & Dr. R. Swathy · Department of Networking and Communications, School of Computing

Prepared October 2026

Contents

1. **Part A: The project**
    1. The project in one page
    2. How VERITAS works, step by step (with a real example)
    3. How every number is calculated
    4. Generative AI, not agentic AI
    5. Is it a RAG project?
2. **Part B: Every concept we used**
    1. Machine Learning (ML)
    2. Deep Learning (DL)
    3. Natural Language Processing (NLP)
    4. Transformers
    5. Generative AI (GenAI)
    6. Supporting technologies
3. **Part C: The research**
    1. Why research: the Review 1 comment
    2. The five research questions
    3. RQ2: is our checker reliable?
    4. RQ4: does the prompt matter?
4. **Part D: The software we built**
    1. Architecture: backend and frontend
    2. Problems we found and fixed
    3. How to run it
    4. Limitations and future work
5. **Part E: The Review 2 presentation, slide by slide**
6. **Part F: Viva preparation**: question bank, glossary, numbers to remember

PART A

The project

What VERITAS is, how it works, and how it produces its numbers.

1. The project in one page

The problem

Generative AI tools such as ChatGPT sometimes produce information that is **false or not backed by any source**. This is called **hallucination**. The dangerous part is that the AI sounds **just as confident** when it is wrong as when it is right.

Even tools that show citations rarely check whether the cited source *actually says* what the AI claims. And when two sources disagree, the AI usually picks one side silently. So the user cannot tell **which sentences to trust**.

OUR PROBLEM STATEMENT (SLIDE 3)

"Generative AI can produce **inaccurate or unsupported information** with high confidence. Existing AI research tools often lack **claim-level verification** and **conflict detection**, making it difficult to assess the reliability of generated information."

Our solution: VERITAS

VERITAS (Latin for *truth*) is a system that:

1. takes the **documents you upload** (research papers, reports, notes);
2. answers your question **using only those documents**, with a source tag on every sentence;
3. then **checks every single fact in its own answer** against the documents;
4. and gives each fact a **confidence score** and a label: ✓ Verified ! Uncertain ✕ Unsupported ⊘ Contradicted.

EASY ANALOGY

Imagine a student who writes an exam answer, then opens the textbook and **ticks or crosses every line of their own answer** before handing it in. VERITAS is that student. The "writing" is done by one AI (the LLM), and the "checking" is done by a different AI (the NLI model).

ONE-LINE PITCH

"We don't try to stop the AI from hallucinating. We **catch it, claim by claim**, and show the user how much to trust each part of the answer."

Who would use it

- **Students and researchers** reading many papers who need answers they can cite.
- **Anyone working with important documents** (policy, medical, legal) where a wrong sentence is costly.
- **Teachers and reviewers** who want to see where an AI answer comes from.

2. How VERITAS works, step by step

We follow one real example through the whole system. You upload the famous paper **"Attention Is All You Need"** and ask: "*What are the main findings of this document?*"

1 **Upload and index (happens once per document).** The PDF's text is extracted page by page. The bibliography (list of references) is removed. The text is cut into passages of about 200 words, called **chunks**; this paper gives 32. Each chunk is turned into **384 numbers** (an **embedding**) that capture its meaning, and stored in a **vector database** (ChromaDB).

▼

2 **Search (retrieval).** Your question is also turned into 384 numbers. The database finds the **20 chunks closest in meaning**. A second, more careful model (a **cross-encoder re-ranker**) re-reads the question together with each chunk and keeps the **best 5**. These are labelled **S1–S5**. The first page (abstract) of the selected document is always included as a candidate, which helps broad questions like "main findings".

▼

3 **Generate the answer (the Generative AI step).** The LLM (**gpt-oss-120b**) receives a carefully engineered prompt: "*You are VERITAS, a meticulous research assistant… Use ONLY these passages. End every factual sentence with its source tag like [S3]. If the evidence is insufficient, say so.*" plus the question and the 5 passages. It writes, for example: "*…the big Transformer achieves 28.4 BLEU, surpassing all previous models by more than 2 BLEU [S3].*"

▼

4 **Split into claims.** The LLM breaks its own answer into **atomic claims**, meaning single checkable facts, and keeps each one's citation. Example: "*The big Transformer achieves 28.4 BLEU on English–German*" → *S3*. Sentences like "the evidence is insufficient to determine X" are not facts, so they are skipped.

▼

5 **Check every claim (NLI verification).** A separate AI model, **DeBERTa-v3**, performs **Natural Language Inference**. It reads a passage (the *premise*) and a claim (the *hypothesis*) and outputs three probabilities: **entail** (the passage supports the claim), **neutral** (it doesn't say), and **contradict** (it says the opposite). Each claim is checked against every passage, both as a whole and in small 3-sentence windows, and the best-supporting match is kept.

▼

6 **Self-consistency check (only in "Deep check" mode).** The LLM answers the same question **4 more times** with more randomness (temperature 0.8). DeBERTa checks whether each of those answers also contains the claim. A fact the AI really got from the documents **keeps coming back**; a guessed fact **changes or disappears**.

▼

7 **Score, label, correct once, report.** Each claim gets a confidence score (Section 3) and a label. In Deep check mode, claims labelled unsupported or contradicted are sent back to the LLM **exactly once** to be corrected or removed using the evidence. The final report shows the answer, every claim with its label and score, the sources, and any conflicts between sources.

TWO MODES IN THE APP

Quick check (about 10–40 seconds): steps 1–5 and 7, without self-consistency or correction.
Deep check (about 20 seconds to 2 minutes): all steps, including 4 extra samples and one correction pass.

Real result for this example

For "main findings" on the Attention paper, VERITAS produced **12 claims: 11 verified and 1 unsupported**. The unsupported one, "a 4-layer model achieves 91.3 F1 in the WSJ-only setting", is actually true, but the number appears only inside a **table**, and PDF tables come out as scrambled text, so the checker could not confirm it. This is an honest example of a known limitation (Part D).

3. How every number is calculated

The results screen shows numbers for every **claim** and for the **whole answer**. Here is exactly where each one comes from.

3.1 Numbers for each claim (on each claim card)

Number	What it means	How it is calculated
Evidence (NLI) green bar	How strongly the documents support the claim.	The **highest "entail" probability** DeBERTa gives this claim across all passages and windows.
Contradiction red bar	How strongly a passage says the opposite.	If some passage clearly supports the claim (entail ≥ 0.5): **that passage's** contradiction. Otherwise: the highest contradiction from any passage.
Self-consistency dark bar, Deep check only	Whether the AI repeats this fact when asked again.	The **average "entail" probability** of the claim across the 4 extra answers.
Confidence big number	Overall trust in the claim, from 0 to 1.	The formula below.
Label coloured badge	The verdict.	The rules below.

```
Deep check: confidence = 0.6 × Evidence + 0.4 × Self-consistency − 0.5 ×
Contradiction
Quick check: confidence = Evidence − 0.5 × Contradiction
(the result is clipped to the range 0 – 1)
```

Labelling rules, checked in this order:

1. If Contradiction ≥ 0.5 **and** Contradiction > Evidence → ⊘ Contradicted
2. else if confidence ≥ 0.70 → ✓ Verified
3. else if confidence < 0.40 → ✕ Unsupported
4. otherwise → ! Uncertain

Worked examples (real numbers from our runs)

Claim	Evidence	Consist.	Contra.	Calculation	Result
"The big Transformer trains in 3.5 days on eight GPUs" (Quick)	0.99	–	0.00	0.99 – 0 = **0.99**	Verified

"Factored verification variants improve performance" (Deep)	1.00	0.31	0.00	0.6 + 0.124 – 0 = **0.72**	**Verified** (just above 0.70)
"The Transformer achieves 35.2 BLEU" (a **fake** claim we invented to test the checker)	0.09	–	0.66	contradiction 0.66 > evidence 0.09	**Contradicted**

The second example is important: the documents fully support the claim, but the AI repeated it in only 31% of its re-answers, so its confidence drops. **Self-consistency caught uncertainty that NLI alone would have missed.**

3.2 Numbers for the whole answer (top row of the results)

Number	Formula	Example
Claims checked	How many claims the answer was split into.	12
Hallucination rate	(Unsupported + Contradicted) ÷ Claims checked. *Uncertain claims are not counted.*	1 unsupported ÷ 12 = **8%**
Mean confidence	The average of all claims' confidence scores.	0.86
Citation accuracy	Among claims that cite a source: the share where **at least one cited passage** supports the claim (entail ≥ 0.5).	11 of 12 = **92%**
Time	Seconds from clicking the button to the result.	39 s
Coloured bar	The share of claims in each label.	green 11, red 1

EVIDENCE VS. CITATION ACCURACY

A claim can be **true** (some passage supports it) but **cite the wrong passage**. Evidence measures the first; citation accuracy measures the second. A wrong citation still counts as a citation mistake.

HONEST POINT: WHO MEASURES THE HALLUCINATION RATE?

The hallucination rate is measured **automatically by our checker (DeBERTa)**, not by a human. It means "the share of claims our checker could not find support for". Because the checker itself is not perfect (70% accuracy on SciFact; see Part C), some flags are false alarms and a few hallucinations can slip through. **That is exactly why we measured the checker separately**: so we know how far to trust our own numbers.

4. Generative AI, not agentic AI

The review gives **negative marks for agentic AI**. VERITAS is a **generative AI** project, and this is easy to show.

What is agentic AI?

An **AI agent** is a system where the AI **decides what to do next**: it plans steps, chooses tools (web search, code, APIs), takes actions, looks at the result, and **loops** until it decides it is finished. The intelligence is in **planning and acting**.

What is generative AI?

Generative AI means models that **create new content** (text, images, code). The intelligence is in **generating and understanding** content.

Agentic AI does this	VERITAS does this instead
The AI **plans its own steps**	**We wrote the steps in code.** Every question goes through steps 1→7 in the same order.
The AI **chooses and calls tools**	The AI **never chooses anything**. Search is ordinary code we wrote, and it runs every time.
The AI **loops** until it decides it is done	**Single pass.** Even the correction happens exactly once, never in a loop.
The AI **acts on its own**	It only answers the user's question, from the user's documents.
Intelligence is in **deciding and acting**	Intelligence is in the **models**: generating text (LLM), understanding meaning (embeddings), checking facts (NLI), measuring certainty (sampling).

SAY IT LIKE THIS

"VERITAS is a fixed pipeline. The LLM's only job is to generate text, and other models check that text. The LLM never decides what happens next, never picks a tool and never loops. That is generative AI."

WORDS TO AVOID

agent, autonomous, plans, decides, tool use, orchestration, multi-agent, task decomposition, reasoning loop

SAFE WORDS

pipeline, grounded generation, retrieval-augmented generation, verification, fixed steps, single pass, claim extraction

HISTORY: WHY THIS MATTERS FOR US

The original Review 1 deck described a "multi-agent research pipeline" with task decomposition and specialised search agents, which is agentic. We redesigned it into the current generative pipeline: the problem statement stayed the same, and the solution changed. We also considered letting the system search the web on its own and rejected it, because that would be agentic. Users upload their own documents instead.

5. Is it a RAG project?

Partly. RAG (Retrieval-Augmented Generation) is the **foundation**, but it is not the whole project. VERITAS is **RAG plus a verification layer**, and the verification layer is our contribution.

Step	Plain RAG	VERITAS
Upload and index documents	✓	✓
Retrieve relevant passages	✓	✓ plus re-ranking and bibliography removal
LLM writes an answer from the passages	✓ **stops here**	✓ with a citation on every sentence
Split the answer into claims	✗	✓
Check each claim with NLI	✗	✓
Self-consistency check	✗	✓
Confidence score and label per claim	✗	✓
Conflict detection and one correction pass	✗	✓
Research evaluation (SciFact, prompt experiments)	✗	✓

Why RAG alone is not enough: in our RQ4 experiment, with the **same retrieved passages**, a badly prompted LLM still produced 74% unsupported claims. RAG gave it the right evidence and it still made things up. Without verification, nobody would notice.

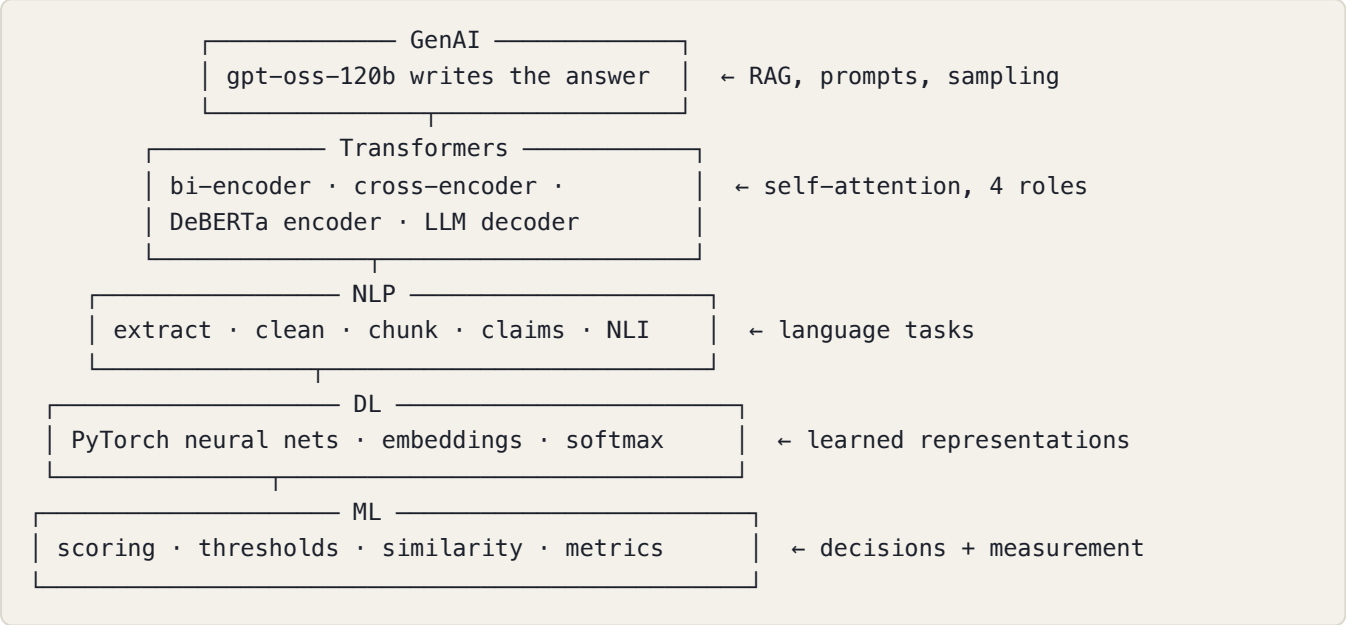
SAY IT LIKE THIS

"VERITAS is a **verified RAG** system. RAG grounds the answer in the user's documents, and our contribution is the claim-level verification layer on top: every claim is checked, scored and labelled, so the user knows exactly which parts to trust."

PART B

Every concept we used

Each concept: what it is in simple words, what exactly we use, and what job it does in VERITAS.



Each layer builds on the one below it. This is the same idea as slide 5 of the presentation.

6. Machine Learning (ML)

In simple words: using data and probabilities to **make decisions**, and **measuring** how good those decisions are.

What we use	What it does in VERITAS	Where
Weighted scoring (a linear combination of signals)	Combines evidence, consistency and contradiction into one confidence score.	`scoring.py`
Decision thresholds	Turn the score into a label (≥ 0.70 verified, < 0.40 unsupported).	`scoring.py`
Nearest-neighbour search with cosine similarity	Finds the chunks whose vectors are closest to the question's vector. Cosine similarity measures the angle between two vectors: 1 means the same direction (same meaning).	ChromaDB

Probabilities	Every model output is a probability, which lets us combine and compare signals.	everywhere
Classification metrics (scikit-learn)	Measure our checker: accuracy, precision, recall, F1, AUROC, confusion matrix (Part C).	`eval/`
Benchmark evaluation	Testing on a labelled public dataset (SciFact) instead of our own opinion.	`eval/`

BE HONEST IF ASKED

The weights (0.6 / 0.4 / 0.5) and thresholds (0.70 / 0.40) were **set by hand**; they were not learned. Good answer: "They're set by hand for now. As future work, we will learn them with logistic regression on SciFact, which also calibrates the confidence score."

ONE LINE

"ML turns the models' probabilities into one confidence score and a decision, and gives us the metrics to prove the system works."

7. Deep Learning (DL)

In simple words: neural networks with many layers that learn rich patterns from huge amounts of data. We use models that were already trained on millions of examples. This is called **transfer learning**: we reuse their knowledge instead of training from scratch.

What we use	What it does in VERITAS
PyTorch	The deep-learning library that runs all our neural networks.
all-MiniLM-L6-v2 6 layers, ~22M parameters	Turns any text into **384 numbers** (an embedding). Similar meaning gives similar numbers, so we can search by **meaning**, not exact words. "Car" and "automobile" end up close together.
ms-marco-MiniLM cross-encoder	A neural ranker trained on Bing search data (MS MARCO). It scores how well each passage answers the question.
DeBERTa-v3-base ~184M parameters	A neural classifier trained on three NLI datasets (MNLI, FEVER, ANLI). It outputs support / neutral / contradict probabilities.
Softmax	The last step of a classifier: it turns raw scores into probabilities that add up to 1.
GPU acceleration (Apple MPS)	Runs DeBERTa on the Mac's graphics chip. We measured 0.26 s per check on the GPU vs 1.67 s on the CPU (**6.4× faster**) with identical outputs.

ONE LINE

"Deep learning gives us the neural networks: embeddings to find evidence by meaning, and a neural classifier to check claims."

8. Natural Language Processing (NLP)

In simple words: techniques that let computers **read, split and understand human text**.

What we use	What it does in VERITAS	Where
PDF text extraction (PyMuPDF)	Pulls text out of each page and keeps page numbers for citations.	`ingest.py`
Text cleaning	Re-joins words split by hyphens at line ends, removes in-text markers like [12], fixes spacing.	`ingest.py`
Bibliography removal	Drops reference lists. They are full of paper titles, so they would match almost any question.	`ingest.py`
Sentence segmentation	Splits text into sentences.	`ingest.py`
Chunking with overlap	Cuts documents into ~220-word passages that overlap by ~40 words, so no idea is cut in half.	`ingest.py`
Tokenization	Splits text into sub-word pieces ("tokens") that the neural models read.	inside each model
Semantic search	Finds passages by meaning rather than exact words.	`retrieve.py`
Claim decomposition	Splits the AI's answer into single, self-contained, checkable facts.	`claims.py`
Natural Language Inference (NLI)	The core NLP task: does text A **support**, **contradict**, or **say nothing about** text B?	`verify.py`
Sentence-window checking	Checks each claim against small 3-sentence windows as well as whole passages, so one key sentence isn't lost in a long passage. (This idea comes from SummaC, Laban et al., 2022.)	`verify.py`
Citation parsing	Reads tags like [S1], (S2) or 【S3】 and links each claim to its source.	`generate.py`

"CLAIM DECOMPOSITION" IS NOT AGENTIC TASK DECOMPOSITION

Claim decomposition splits **text** into facts. It is an NLP step, done at one fixed point in the pipeline. Agentic task decomposition means an AI **planning actions**. They only share a word.

ONE LINE

"NLP prepares the text (extract, clean, chunk) and performs the key language task, NLI, which decides whether each claim is supported."

9. Transformers

In simple words: the Transformer is the neural-network design behind all modern AI (introduced in "Attention Is All You Need", 2017). Its key idea is **self-attention**: when reading a word, the model looks at every other word to understand the context. In "the bank of the river", attention links "bank" to "river", so the model knows it isn't a money bank.

We use the Transformer in **four different roles**:

Role	Model	How it reads	Its job in VERITAS
Bi-encoder	MiniLM	Reads the question and each passage **separately** and turns each into a vector.	Fast search across thousands of passages.
Cross-encoder	ms-marco MiniLM	Reads question + passage **together**, so attention links words across both.	Careful re-ranking (top 20 → top 5).
Encoder classifier	DeBERTa-v3	Reads passage + claim together, then classifies.	**Checks every claim** (NLI).
Decoder (LLM)	gpt-oss-120b	**Generates** text one token at a time.	Writes the answer, splits claims, explains verdicts.

ENCODER VS. DECODER

An **encoder understands**: it reads text and outputs a judgement or a vector. A **decoder writes**: it produces new text word by word, each word predicted from the words before it (this is called **autoregressive generation**).

BI-ENCODER VS. CROSS-ENCODER: WHY USE BOTH?

A bi-encoder is **fast**: passage vectors are computed once and searched instantly, but it never sees question and passage together. A cross-encoder is **accurate** because it reads them together, but too slow to run on every passage. So we use the fast one to get 20 candidates and the accurate one to pick the best 5. This is called **two-stage retrieval**.

10. Generative AI (GenAI)

In simple words: AI that **creates new content**. In VERITAS, that content is text: the grounded answer.

What we use	What it does in VERITAS

Large Language Model: gpt-oss-120b	An open-weight LLM with 120 billion parameters, run in the cloud through the Groq API. It generates the answer.
Retrieval-Augmented Generation (RAG)	The retrieved passages are inserted into the prompt, so the answer is **grounded** in your documents instead of the model's memory.
Temperature sampling	Temperature controls randomness. **0.1** is nearly deterministic (the main answer). **0.8** gives varied answers (self-consistency samples).
Self-consistency (the SelfCheckGPT method, Manakul et al., 2023)	Generate several answers; facts that keep coming back are trusted more. Hallucinated details tend to change between samples.
Prompt engineering	Carefully designed prompts of 8 types (Section 10.1). We showed experimentally that prompt design alone changes the hallucination rate from 74% to 0% (Part C).
Structured output (JSON mode)	Makes the LLM return claims in a fixed, machine-readable format.
Single-pass correction	Weak claims are sent back once to be rewritten from the evidence. It runs once, not in a loop.
LLM-as-judge	A baseline in which the LLM itself classifies claims, to compare against DeBERTa.

10.1 The eight prompt types (from the class activity) and where we use each

Prompt type	Used for	The actual prompt (shortened)
Role-play	System persona	"You are VERITAS, a meticulous research assistant. You never state a fact that is not supported by the evidence you are given, you cite the source of every factual sentence, and you clearly say when the evidence is insufficient or when sources disagree."
Instructional	Grounded answer	"1. Use ONLY the evidence passages below. 2. End every factual sentence with its source tag(s), e.g. [S1]. 3. If the evidence does not answer part of the question, write 'The provided evidence is insufficient to determine…'. 4. If passages disagree, report both positions. 5. At most 8 sentences."
Completion + Few-shot	Claim extraction	"Break the answer below into atomic factual claims… Example → Output: {"claims": [{"claim": "Aspirin lowered heart-attack risk by 22% in men.", "citations": ["S1"]}…]}". One worked example, then the LLM completes the same format.
Classification	LLM-as-judge baseline	"Classify whether the evidence supports the claim… Reply in JSON: SUPPORTED | CONTRADICTED | NOT_ENOUGH_INFO."
Reasoning (chain-of-thought)	Explaining a verdict	"Explain step by step, in at most 3 short sentences for a non-expert, why this claim received its verdict…"

Comparison	Conflict explanation	"Two sources take different positions on the same claim… Compare what each one says, and a plausible reason they differ. Do not decide who is right."
Information	Closed-book baseline	"Answer the following research question accurately and concisely." No documents are given; used to compare against RAG.
Creative	Lay summary	"Rewrite these verified findings as a short, engaging explanation for a curious 15-year-old. Use one everyday analogy… do not add any new facts."

All prompts are in `veritas/prompts.py` and visible in the app's **Prompt Lab** page.

ONE LINE

"GenAI is the core: the LLM generates a grounded, cited answer, and we use sampling and prompt engineering to measure and reduce hallucination."

11. Supporting technologies

Category	What	Why we use it
Vector database	**ChromaDB**	Stores passage vectors and finds the closest ones quickly, using cosine similarity.
Model hub	**Hugging Face**	Where MiniLM, the cross-encoder and DeBERTa are downloaded from (free, open models).
LLM provider	**Groq API** (free tier)	Runs the 120B-parameter LLM in the cloud. Limits: 1,000 requests/day and 8,000 tokens/minute.
Benchmark dataset	**SciFact** (Wadden et al., 2020)	5,183 scientific abstracts with expert-labelled claims, for objective evaluation.
Demo documents	7 open-access papers on LLM hallucination	SelfCheckGPT, FActScore, RAG (Lewis), Hallucination Survey (Ji), Chain-of-Verification, TruthfulQA, Fine-tuning on New Knowledge.
Response cache	Saves every LLM reply to disk	Repeated questions are instant, results are reproducible, and we stay within free limits.
Backend	**FastAPI** (Python)	The web API: upload, ask, job progress, explain, prompts, evaluation results.
Frontend	**React + Vite + Tailwind CSS**	The website: one workspace to upload a PDF, ask, and see verified claims.
Prototype UI	Streamlit	The first quick interface, kept as a backup.

Languages	Python, TypeScript	Python for the AI and backend; TypeScript for the website.

PART C

The research

How we turned the project into a study with measurable results, as the Review 1 panel asked.

12. Why research: the Review 1 comment

REVIEW 1 COMMENT

"Look at the project from a research perspective. Don't only build the system; research it and do more analysis."

Our response, shown on slide 2:

1. **Reframed as a research study:** five research questions, each with an experiment and a metric.
2. **Standard benchmark:** SciFact, so our numbers are comparable with published work, not invented by us.
3. **Baselines and ablations.** A **baseline** is what you compare against, such as a plain LLM or a different checker. An **ablation** means switching one part off to see how much it matters.
4. **Quantitative analysis:** accuracy, F1, AUROC and hallucination rate, instead of saying "it works well".

13. The five research questions

RQ	Question	Experiment	Status
RQ1	Does grounding (RAG) reduce hallucination compared with a plain LLM?	Closed-book LLM vs RAG vs RAG + VERITAS on the same questions	Script ready, full run next
RQ2	Can a transformer NLI model reliably verify scientific claims?	DeBERTa vs expert labels on SciFact	**Done: results on slide 6**
RQ3	Does self-consistency add signal beyond NLI?	Ablation: NLI only vs consistency only vs both	Script ready
RQ4	Which prompt design hallucinates least?	Same questions and evidence, 5 prompt designs	**Preview done: slide 7**
RQ5	Do model size and retrieval settings matter?	gpt-oss-20b vs 120b; re-ranker on/off; top-k	Planned

14. RQ2: is our checker reliable?

Why this matters: we measure the AI's hallucinations with our checker (DeBERTa). If the checker were bad, all our numbers would be meaningless. So first we test the checker itself against **human experts**.

> **ANALOGY: THE THERMOMETER**
>
> The **hallucination rate** is like a thermometer reading. **RQ2** is like checking the thermometer against a lab-grade one. We don't compare the two numbers to each other; RQ2 tells us **how much to trust** the hallucination rate.

How the experiment works

1. SciFact gives **340 (claim, abstract) pairs** from its development set. Scientists labelled each one: **Supported** (138), **Contradicted** (71) or **Not enough info** (131).
2. DeBERTa reads each pair and picks the label with the highest probability.
3. We compare DeBERTa's labels with the experts' labels.

The confusion matrix

Rows are what the experts said; columns are what DeBERTa said. The diagonal (bold) shows agreements.

Experts ↓ / DeBERTa →	Supported	Contradicted	Not enough info	Total
Supported	**95**	12	31	138
Contradicted	12	**42**	17	71
Not enough info	20	10	**101**	131

The metrics, explained

Metric	What it means	Our result
Accuracy	Share of pairs where DeBERTa agrees with the experts. (95 + 42 + 101) ÷ 340.	**0.70** (random guessing among 3 labels ≈ 0.33)
Precision	"When it says X, how often is it right?"	Supported 0.75 · Contradicted 0.66 · NEI 0.68
Recall	"Of all real X, how many did it find?"	Supported 0.69 · **Contradicted 0.59** · NEI 0.77
F1	The balance (harmonic mean) of precision and recall.	0.72 · 0.62 · 0.72
Macro-F1	The average F1 across the 3 labels, so a weak label can't hide behind a strong one.	**0.69**

AUROC	Pick one truly supported claim and one not-supported claim at random: how often does the supported one get the higher support score? 0.5 = coin flip, 1.0 = perfect.	**0.86**

Analysis

- The checker clearly works: 0.70 accuracy vs 0.33 random, and 0.86 AUROC.
- The most common mistake is **cautious**: 31 supported claims were labelled "not enough info". For a hallucination detector, under-trusting is the safer mistake.
- **Contradictions are the hardest** (recall 0.59). This is the main target for improvement: fine-tuning DeBERTa on SciFact's training set.
- When the checker says "supported", it is right about **3 times out of 4** (precision 0.75), so a few hallucinations can slip through as "verified".

15. RQ4: does the prompt matter?

Setup: 3 questions about the demo papers. For each question the **same 5 passages** were retrieved once. Then 5 different prompt designs generated answers from those identical passages, and every claim was checked by DeBERTa. **Only the prompt changed.**

Results (shown on slide 7)

Prompt design	Claims	Hallucination rate	Citation accuracy
✗ Bad prompt (vague: just question + passages)	39	74%	29%
Zero-shot instructions	13	8%	77%
Role + instructions (VERITAS default)	15	0%	87%
Few-shot	12	17%	75%
Chain-of-thought	6	0%	100%

Where 74% and 0% come from

Bad prompt		
Question	Claims	Flagged
Detect hallucinations?	21	16 ✕ + 3 ⊘
Does RAG reduce it?	10	4 ✕
Does fine-tuning increase it?	8	5 ✕ + 1 ⊘

Our prompt		
Question	Claims	Flagged
Detect hallucinations?	6	0 (1 uncertain)
Does RAG reduce it?	5	0 (1 uncertain)
Does fine-tuning increase it?	4	0

Total	**39**	**29 → 74%**

Total	**15**	**0 → 0%**

Why the difference is so big

The **bad prompt** gets the passages but **no rules**, so the LLM writes a long essay from its **own memory**. Its answer literally begins: "*…a concise, citation-rich overview that pulls together the most-cited detection approaches… and a few complementary papers.*" It adds methods, numbers and papers that are not in the 5 passages, and the checker flags them.

Our prompt gives a **role** (a careful assistant who never states unsupported facts) and **rules** (only the evidence, cite every sentence, admit gaps, at most 8 sentences). The answers stay short and inside the passages.

WHAT "0%" MEANS AND DOES NOT MEAN

It means: across these 3 questions, the checker found support for all 15 claims.

It does not mean "our AI never hallucinates". Reasons: (1) it is a small 3-question preview; (2) the checker isn't perfect (precision 0.75); (3) it isn't always 0%: the same prompt gave 8% on the Attention paper. Chain-of-thought's 0% comes with very short answers (6 claims), which is a safety vs. completeness trade-off.

SAY IT LIKE THIS

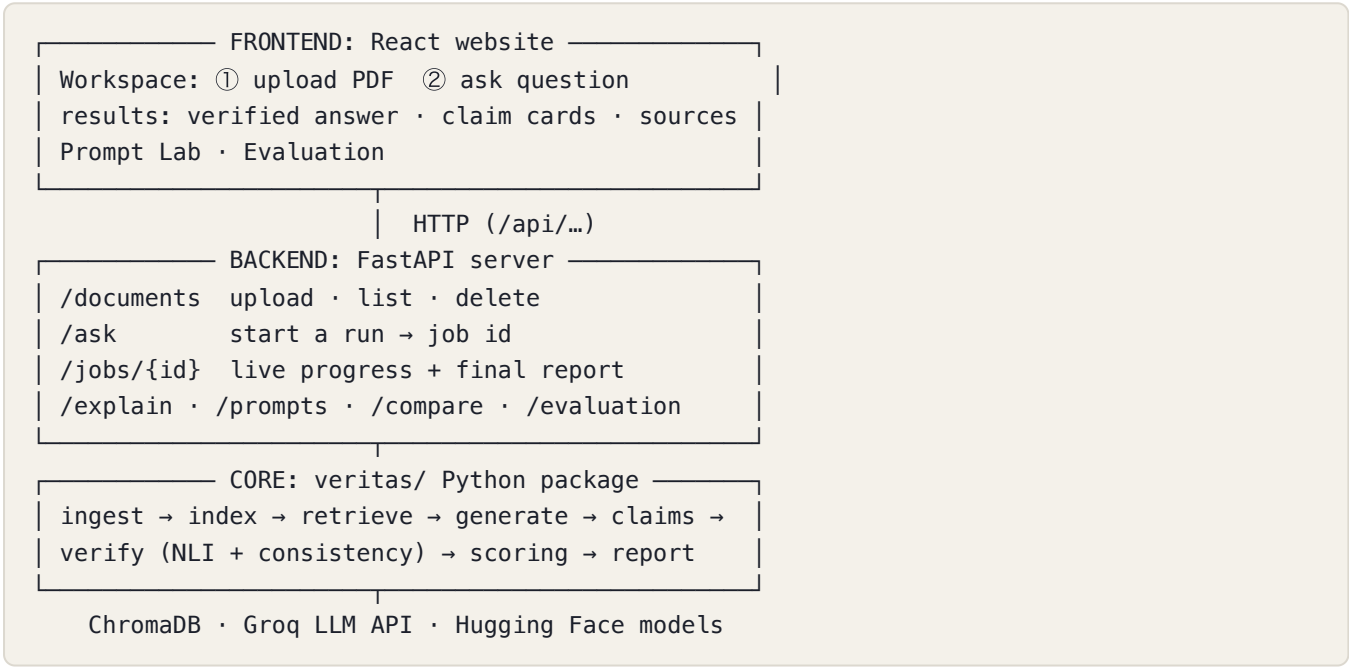
"With identical evidence, changing only the prompt reduced the measured hallucination rate from 74% to 0% on our 3-question preview: 29 of 39 claims flagged versus 0 of 15. The rate is measured by our NLI checker, which we validated separately against expert labels on SciFact."

PART D

The software we built

The architecture, the problems we found and fixed, and how to run it.

16. Architecture: backend and frontend



The code, file by file

File	What it does
`veritas/config.py`	Every setting in one place: models, chunk size, top-k, temperatures, weights, thresholds.
`veritas/ingest.py`	Reads PDF/DOCX/TXT/MD, cleans text, removes bibliographies, splits into chunks.
`veritas/index.py`	Embeds chunks and stores and searches them in ChromaDB.
`veritas/retrieve.py`	Two-stage search: dense search, then cross-encoder re-ranking, with source diversity.
`veritas/llm.py`	The Groq LLM client with an on-disk cache and rate-limit handling.
`veritas/prompts.py`	Every prompt, labelled by prompt type.

`veritas/generate.py`	Grounded answer, closed-book baseline, self-consistency samples, citation normalisation.
`veritas/claims.py`	Claim extraction (LLM in JSON mode, with a rule-based fallback).
`veritas/verify.py`	The NLI checker: whole-passage and sentence-window checks, plus self-consistency.
`veritas/scoring.py`	The confidence formula, labels and conflict detection.
`veritas/report.py`	Report structure, summary statistics, Markdown export.
`veritas/pipeline.py`	Runs all steps in the fixed order, with progress messages.
`api/main.py`	The FastAPI backend: runs questions as queued background jobs, one at a time.
`web/src/`	The React frontend: Workspace, Prompt Lab and Evaluation pages.
`eval/`	Experiment scripts: `rq2_verifier.py`, `prompt_compare.py`, `rq_generation.py`.

Why one job at a time? The GPU checker and the free Groq rate limit are shared resources. Questions wait in a queue, and the user sees their position.

17. Problems we found and fixed

Finding and fixing these is part of the research story, and good material for the viva.

Problem	What was wrong	Fix
Checking was very slow	DeBERTa ran on the CPU: 1.67 s per check; one RQ2 run didn't finish in 25+ minutes.	Run it on the Mac's GPU: **6.4× faster**, same outputs. RQ2 then took 1 min 39 s.
True claims marked "contradicted"	NLI scores a passage that simply *doesn't mention* a fact as "contradiction". The first scoring took the strongest contradiction from *any* passage.	If a passage clearly supports the claim, judge by **that** passage. Real cross-document disagreement goes to conflict detection.
Meta-sentences counted as claims	"The evidence does not give…" was being checked as a fact.	A filter skips statements about the evidence itself.
Search matched bibliographies	Reference lists are full of paper titles, so they matched almost any question.	Remove everything after the last "References" heading, except prose such as appendices.
Long passages diluted facts	A key sentence inside a 220-word passage (often with tables) was missed.	Also check 3-sentence windows and keep the best match. Every one of 8 test claims (5 true, 3 fake) was then judged correctly.

Different citation styles	The LLM sometimes wrote 【S3】 or (S2, S5) instead of [S3].	All styles are converted to [S3] automatically.
Answers mixed with other papers	A question about a newly uploaded PDF also searched the 7 demo papers.	Document selection: search only inside the selected documents.

NOTE ON THE SLIDE NUMBERS

The numbers in the Review 2 deck were measured **before** the last three fixes. They are valid results for that version. If asked, say: "Since then we have improved retrieval and verification, and we will re-run the experiments for the next review."

18. How to run it

```
# one command: builds the website and serves website + API
cd /Users/ompathania/Desktop/gen_ai_1
./start.sh                      # then open http://localhost:8000

# or, while developing, two terminals:
source .venv/bin/activate
uvicorn api.main:app --reload --port 8000     # terminal 1: backend
cd web && npm run dev                          # terminal 2: open http://localhost:5173

# experiments
python -m eval.rq2_verifier          # RQ2 (about 2 minutes on GPU)
python -m eval.prompt_compare        # RQ4 preview
```

Using the website: ① upload a PDF (it is selected automatically) → ② type a question, or click a suggested one → choose **Quick check** or **Deep check** → click **Get verified answer**. Click any [S2] tag to highlight that source; click a claim card to see its evidence, per-source scores, and an "Explain this verdict" button.

IF SOMETHING GOES WRONG

"Address already in use" → run `pkill -f "uvicorn api.main"` and start again. "Cannot reach the server" → the backend isn't running. The first question is slower (~15–20 s) because the models load once.

19. Limitations and future work

Limitation	Future work
Contradiction detection is the weakest point (recall 0.59).	Fine-tune DeBERTa on the SciFact training set.

Scoring weights and thresholds are set by hand.	Learn them with logistic regression; check calibration with a reliability diagram.
Tables and figures in PDFs come out as scrambled text.	Table extraction, or a vision model that describes figures and tables (multi-modal prompts).
RQ4 is a 3-question preview.	Full RQ1/RQ3/RQ4/RQ5 runs on 20–50 SciFact questions.
Free tier: 8,000 tokens/minute, shared by all users.	Queueing (done); a paid tier or smaller models for public deployment.
English only; text-based PDFs only (no scanned images).	Multilingual NLI models; OCR for scanned PDFs.
Runs locally.	Deployment on Hugging Face Spaces (Docker, free CPU tier).

PART E

The Review 2 presentation, slide by slide

For each slide: what it shows, what every term means, what to say, and what might be asked. The review is strictly 5 minutes, so the time budget is given for each slide.

THE REVIEW 2 REQUIREMENTS

Continue in the Review 1 template, add a slide with the Review 1 comments, then three content slides:

1. Justification on ML/DL/NLP/Transformer → GenAI concepts (**10 marks**)
2. Generated content and analysis linked with objectives, with the refined pipeline diagram (**5 marks**)
3. Prompt engineering on use-case scenarios (**5 marks**)

Negative marking for agentic AI concepts.

Slide 1: Title (≈10 seconds)

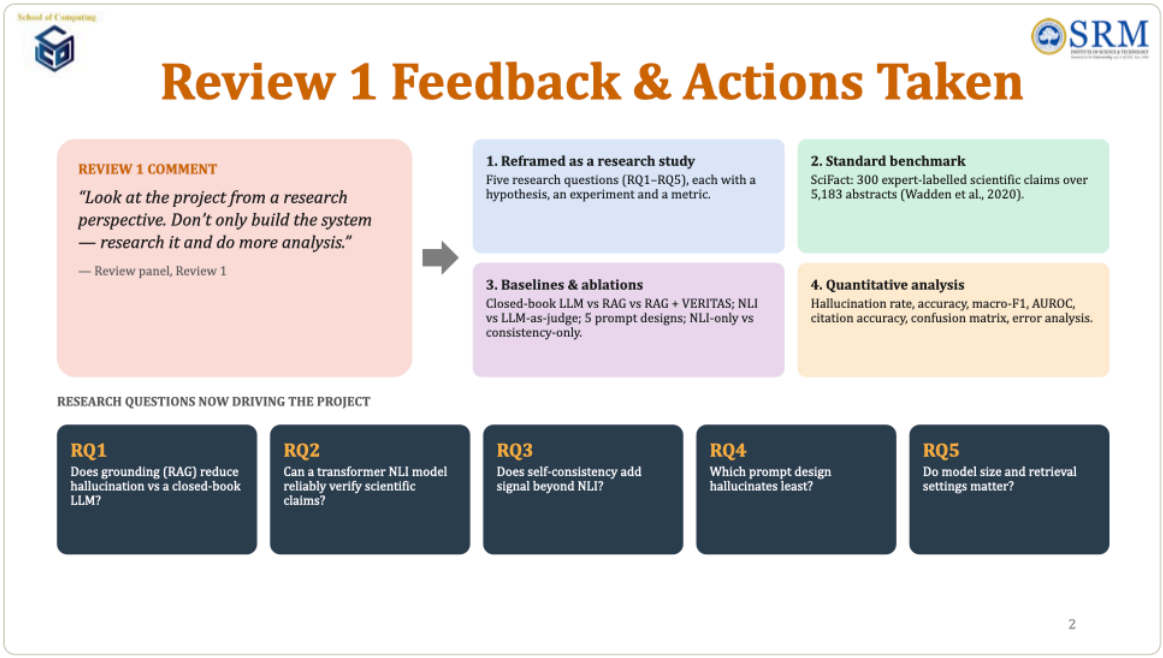


What it shows: course code (21CSE306P Applied Generative AI), project name, team table, faculty, the UN Sustainable Development Goal 9 badge (Industry, Innovation and Infrastructure).

WHAT TO SAY

"Good morning. Our project is VERITAS: Evidence-Driven Generative AI for Reliable Research."

Slide 2: Review 1 Feedback & Actions Taken (≈30 seconds)



Part by part

- **Pink box (left):** the Review 1 comment: research it and do more analysis, don't only build.
- **Arrow:** what we did about it.
- **Four action boxes:**
  1. **Reframed as a research study:** five research questions, each with a hypothesis, an experiment and a metric.
  2. **Standard benchmark:** SciFact, 300 expert-written claims over 5,183 abstracts. "Wadden et al., 2020" is the paper that published it.
  3. **Baselines and ablations:** closed-book LLM vs RAG vs RAG + VERITAS; NLI vs LLM-as-judge; 5 prompt designs; NLI-only vs consistency-only.
  4. **Quantitative analysis:** the metrics we now report.
- **Bottom row (RQ1–RQ5):** the five research questions from Section 13.

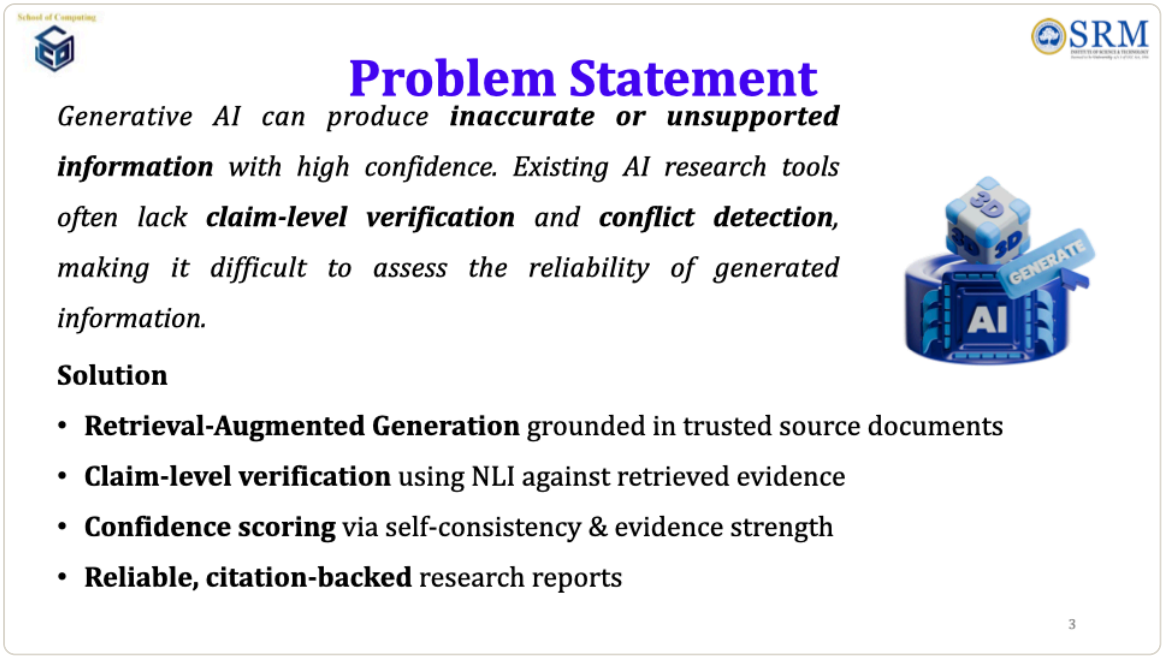
WHAT TO SAY

"In Review 1 the panel asked us to treat VERITAS as research, not only as a system to build. We reframed it around five research questions, adopted the SciFact benchmark, added baselines and ablations, and now report quantitative metrics. Today I'll present results for RQ2 and a preview of RQ4."

POSSIBLE QUESTION

"What is a baseline?" What we compare our system against, to show improvement: for example, an LLM with no documents.

Slide 3: Problem Statement (≈15 seconds)



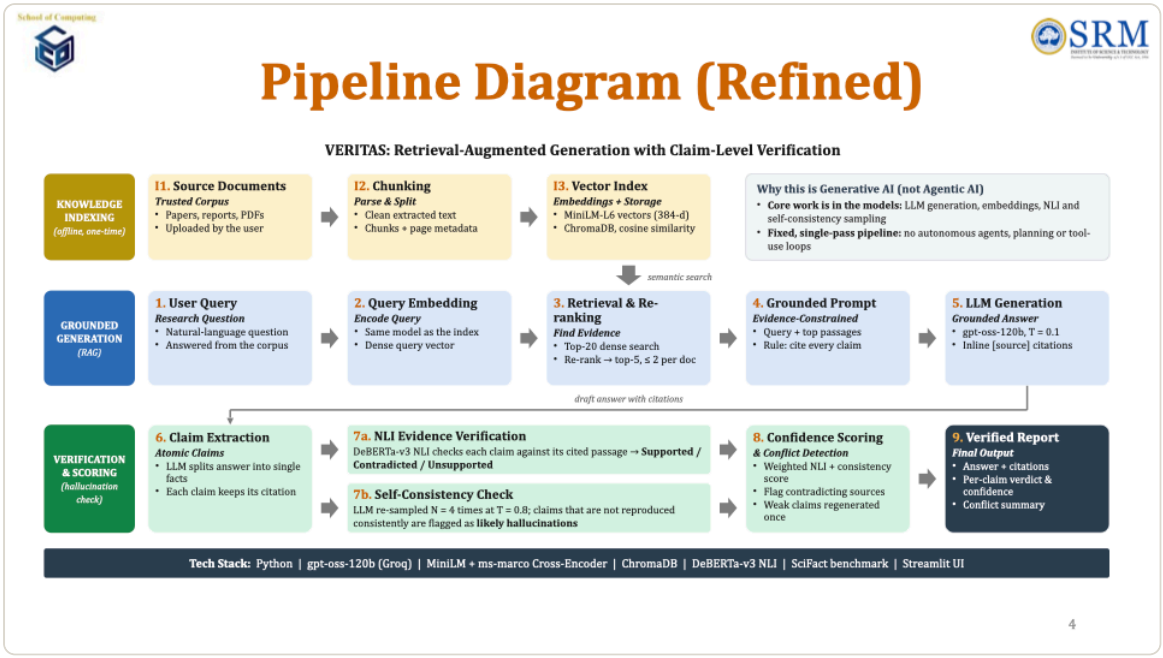
What it shows: the unchanged problem statement from Review 1, and four solution bullets:

- **Retrieval-Augmented Generation** grounded in trusted source documents (Section 10)
- **Claim-level verification** using NLI against retrieved evidence (Section 8)
- **Confidence scoring** via self-consistency and evidence strength (Section 3)
- **Reliable, citation-backed** research reports

WHAT TO SAY

"The problem is unchanged: LLMs produce unsupported information with high confidence, and current tools don't check individual claims or detect conflicts between sources. Our solution is grounded generation followed by claim-level verification."

Slide 4: Pipeline Diagram (Refined) (≈40 seconds)



"Refined" means the diagram now shows the **actual models and settings we implemented**. It has three rows (lanes):

Row 1: Knowledge Indexing (gold, offline, once per document)

- **I1 Source Documents:** papers, reports, PDFs uploaded by the user.
- **I2 Chunking:** clean the extracted text, split into chunks with page metadata.
- **I3 Vector Index:** MiniLM-L6 vectors (384 numbers each), stored in ChromaDB with cosine similarity.

Row 2: Grounded Generation (blue, RAG)

- **1 User Query → 2 Query Embedding** (the same MiniLM model turns the question into a vector)
- **3 Retrieval & Re-ranking:** top-20 dense search, then the cross-encoder re-ranks to the top 5, at most 2 per document (so several sources are seen).
- **4 Grounded Prompt:** question + top passages + the rule "cite every claim".
- **5 LLM Generation:** gpt-oss-120b at temperature 0.1 writes the answer with inline [S#] citations.

Row 3: Verification & Scoring (green)

- **6 Claim Extraction:** the LLM splits the answer into atomic claims; each keeps its citation.
- **7a NLI Evidence Verification:** DeBERTa-v3 checks each claim against its cited passage → Supported / Contradicted / Unsupported.
- **7b Self-Consistency Check:** the LLM is re-sampled N = 4 times at temperature 0.8; claims not reproduced consistently are flagged as likely hallucinations.
- **8 Confidence Scoring & Conflict Detection:** the weighted NLI + consistency score; contradicting sources are flagged; weak claims are regenerated once.

- **9 Verified Report:** answer + citations, a verdict and confidence for each claim, and a conflict summary.

Grey box (top right): "Why this is Generative AI (not Agentic AI)". **Dark strip (bottom):** the tech stack.

WHAT TO SAY

"This is the refined pipeline with the models we implemented. There are three stages. Indexing: uploaded documents are chunked and embedded with MiniLM into ChromaDB. Grounded generation: we retrieve the top 20 passages, re-rank them with a cross-encoder down to 5, and gpt-oss-120b writes an answer that cites every sentence. Verification: the answer is split into atomic claims, each claim is checked by DeBERTa-v3 NLI and by self-consistency across 4 re-sampled answers, and then it is scored. It's a fixed, single-pass pipeline, with no agents, planning or tool use."

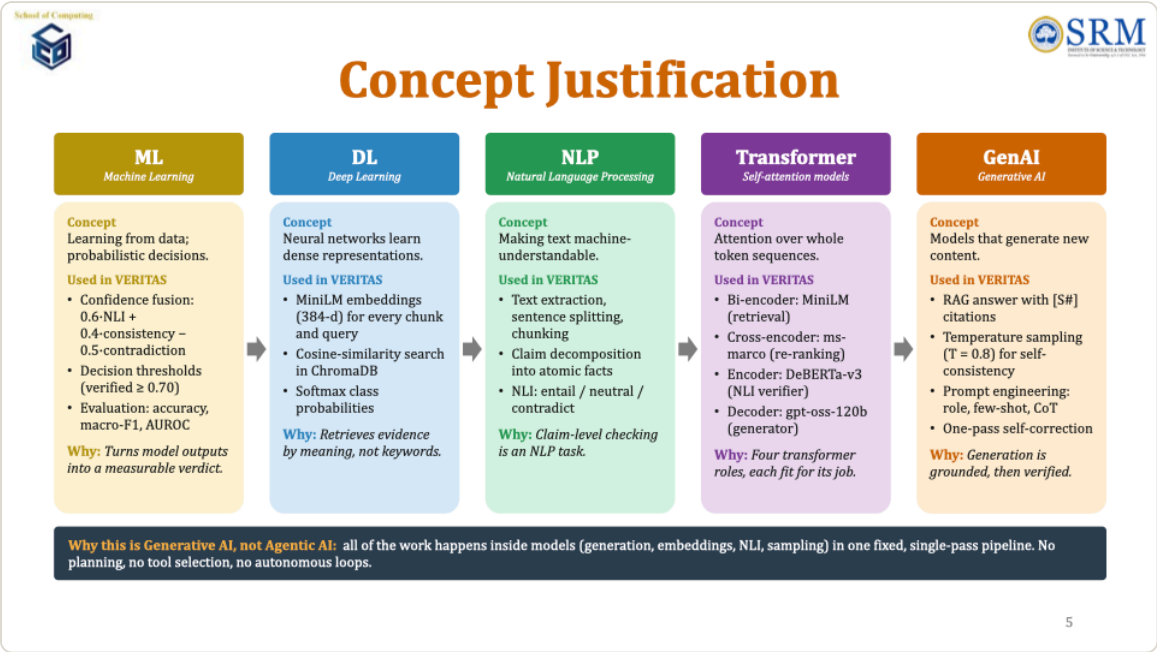
POSSIBLE QUESTIONS

"Why re-rank?" Dense search is fast but rough; the cross-encoder reads question and passage together and is more accurate, but too slow to run on everything. So we use both (two-stage retrieval).

"Why at most 2 passages per document?" So the LLM sees several sources, which is needed for balanced answers and conflict detection.

"Why temperature 0.1 vs 0.8?" 0.1 gives a stable main answer; 0.8 gives varied samples so we can see which facts stay consistent.

Slide 5: Concept Justification (≈70 seconds, worth 10 marks, don't rush)



Five columns, from basic to advanced, with arrows showing that each builds on the previous one. Each column has a **Concept** (what it is), **Used in VERITAS** (where it appears) and **Why** (why we need it). Full details are in Part B.

Column	What each point on the slide means
ML	**Confidence fusion** = combining three signals into one score with weights (0.6 NLI + 0.4 consistency − 0.5 contradiction). **Decision thresholds** = the cut-offs that turn a score into a label (verified ≥ 0.70). **Evaluation** = accuracy, macro-F1 and AUROC (Section 14). *Why:* it turns model outputs into a measurable verdict.
DL	**MiniLM embeddings (384-d)** = a neural network turns each chunk and query into 384 numbers. **Cosine-similarity search** = find the closest vectors in ChromaDB. **Softmax class probabilities** = how a neural classifier outputs probabilities. *Why:* it retrieves evidence by meaning, not keywords.
NLP	**Text extraction, sentence splitting, chunking** = preparing documents. **Claim decomposition** = splitting an answer into atomic facts (a text operation, not agentic planning). **NLI: entail / neutral / contradict** = the language-understanding task at the heart of verification. *Why:* claim-level checking is an NLP task.
Transformer	The same architecture in four roles: **bi-encoder** MiniLM (retrieval), **cross-encoder** ms-marco (re-ranking), **encoder** DeBERTa-v3 (NLI verifier), **decoder** gpt-oss-120b (generator). *Why:* each role is fit for its job.
GenAI	**RAG answer with [S#] citations** = grounded generation. **Temperature sampling (T = 0.8)** for self-consistency. **Prompt engineering**: role, few-shot, chain-of-thought. **One-pass self-correction** of weak claims. *Why:* generation is grounded, then verified.

Bottom strip: why this is Generative AI, not Agentic AI: all the work happens inside models (generation, embeddings, NLI, sampling) in one fixed, single-pass pipeline; no planning, no tool selection, no autonomous loops.

WHAT TO SAY

"Each concept layer builds on the one before. ML: our confidence score is a weighted fusion with decision thresholds, evaluated with accuracy, F1 and AUROC. DL: neural embeddings let us retrieve evidence by meaning, not keywords. NLP: chunking, claim decomposition, and natural language inference, a core NLP task. Transformers: we use them in four roles: a bi-encoder for retrieval, a cross-encoder for re-ranking, an encoder (DeBERTa) for verification, and a decoder (the LLM) for generation. GenAI: retrieval-augmented generation with citations, temperature sampling for self-consistency, prompt engineering, and one-pass self-correction. All the work happens inside models, which is why this is Generative AI and not Agentic AI."

POSSIBLE QUESTIONS

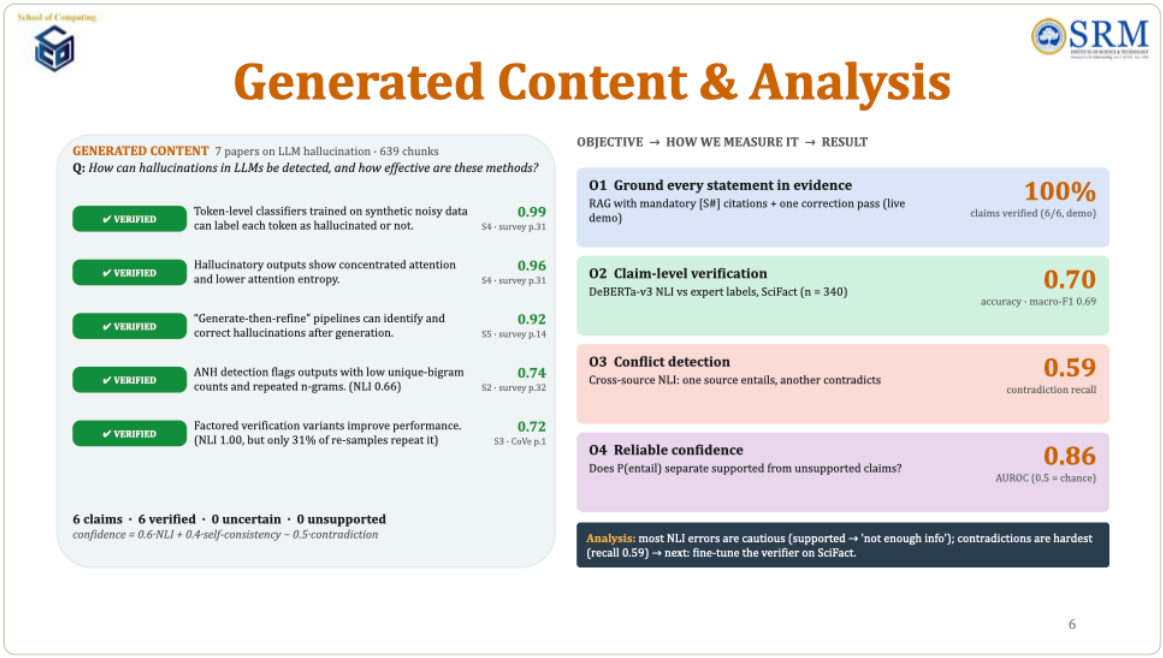
"What is attention?" Each word looks at every other word to understand context; it is the core of the Transformer.

"Encoder vs. decoder?" An encoder reads and understands (classifies, embeds); a decoder generates text word by word.

"Is the ML trained?" The weights and thresholds are set by hand for now; we plan to learn them with logistic regression (be honest).

"What does 384-d mean?" Each text becomes a vector of 384 numbers.

Slide 6: Generated Content & Analysis (≈70 seconds)



Left: generated content (a real system output)

- **Corpus:** 7 papers on LLM hallucination (639 chunks at that time; 575 after the later bibliography fix).
- **Question:** "How can hallucinations in LLMs be detected, and how effective are these methods?"
- **Five claim rows:** each has a verdict badge, the claim, the confidence score, and its source (e.g. "S4 · survey p.31" = passage S4, from the survey paper, page 31).
- **Key talking point, the last row (0.72):** NLI support is 1.00, but only 31% of the re-sampled answers repeat it, so its confidence drops to 0.72. This shows **self-consistency catching uncertainty that NLI alone would miss**.
- **Footer:** 6 claims, 6 verified, 0 uncertain, 0 unsupported, and the confidence formula.

Right: each objective linked to a measured result

Objective	How we measure it	Result	Meaning
O1 Ground every statement	RAG + mandatory citations + one correction pass (live demo)	**100%**	All 6 demo claims verified.
O2 Claim-level verification	DeBERTa vs expert labels, SciFact (n = 340)	**0.70**	Accuracy (random ≈ 0.33); macro-F1 0.69.
O3 Conflict detection	Cross-source NLI: one source entails, another contradicts	**0.59**	Contradiction recall: it catches 59% of real contradictions. Our weakest point.
O4 Reliable confidence	Does P(entail) separate supported from unsupported claims?	**0.86**	AUROC: 0.5 = chance, 1.0 = perfect.

Analysis box: most NLI errors are cautious ones (supported → "not enough info"); contradictions are hardest (recall 0.59); next step: fine-tune the verifier on SciFact.

WHAT TO SAY

"On the left is real output from our system on 7 papers about LLM hallucination. Every claim has a verdict, a confidence score and a source page. Note the last claim: the evidence fully supports it, with NLI at 1.00, but only 31% of re-sampled answers repeat it, so its confidence drops to 0.72. That's self-consistency catching uncertainty NLI alone would miss. On the right, each objective is linked to a measurement. On SciFact our verifier reaches 0.70 accuracy against expert labels, where random would be 0.33, with an AUROC of 0.86. Contradiction recall is 0.59, our weakest point, so our next step is to fine-tune the verifier on SciFact."

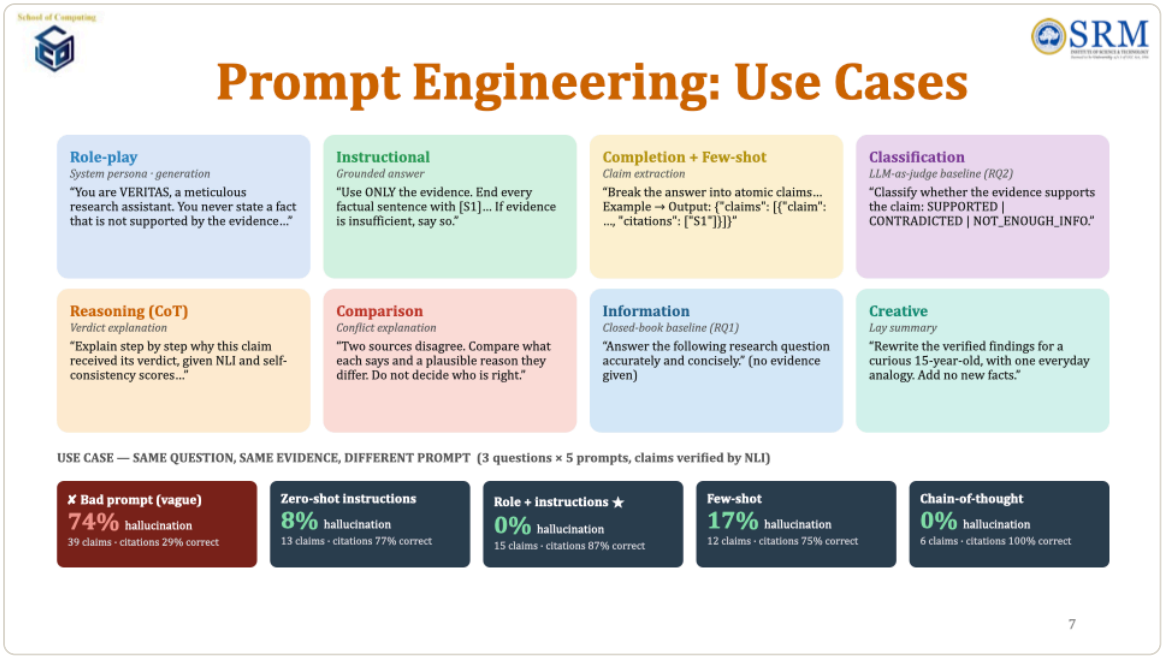
POSSIBLE QUESTIONS

"Why is accuracy only 0.70?" It is a 3-way decision on hard scientific text against whole abstracts; most errors are cautious, which is the safer failure; fine-tuning is next.

"How is 100% computed?" Verified claims ÷ all claims in this demo = 6 ÷ 6.

"What is AUROC?" The chance that a truly supported claim scores higher than a non-supported one.

Slide 7: Prompt Engineering: Use Cases (≈60 seconds)



Top: eight prompt cards

Each card shows the **prompt type** (from the class activity), the **stage** where VERITAS uses it, and a shortened version of the **real prompt**. Full table in Section 10.1:

Role-play (system persona) · Instructional (grounded answer) · Completion + Few-shot (claim extraction) · Classification (LLM-as-judge baseline) · Reasoning/CoT (verdict explanation) · Comparison (conflict explanation) · Information (closed-book baseline) · Creative (lay summary).

Bottom: the use case experiment (RQ4 preview)

Same 3 questions, same evidence, only the prompt changed. Each box shows the hallucination rate, number of claims and citation accuracy:

- **Bad prompt (vague): 74%** (39 claims, citations 29% correct): the red box
- Zero-shot instructions: 8%
- **Role + instructions ★ (our default): 0%** (15 claims, citations 87% correct)
- Few-shot: 17%
- Chain-of-thought: 0%, but only 6 claims (very short answers)

WHAT TO SAY

"We use eight prompt types across the pipeline: role-play for the system persona, instructional for grounding, completion with few-shot for claim extraction, classification for the LLM-judge baseline, reasoning for explanations, comparison for conflicts, information for the closed-book baseline, and creative for lay summaries. The bottom row is the use case: the same questions and the same evidence, with only the prompt changed. A vague prompt hallucinated 74% of the time. Our role-plus-instruction prompt hallucinated 0% on these questions. Prompt design alone removes almost all of the hallucination. Chain-of-thought also reached 0%, but with much shorter answers, so there's a trade-off."

POSSIBLE QUESTIONS

"Why did few-shot do worse (17%)?" We haven't tested the cause yet. A likely reason: our example is from a different domain (vitamin D and fractures), so the model may copy its summarising style rather than stick to the evidence. Testing domain-matched examples is part of the full RQ4 run.

"0% seems too good?" It means 0 of 15 claims flagged on a 3-question preview, measured by our checker; the full run comes next.

"Where is the multi-modal prompt?" Not implemented yet: future work for reading figures and tables.

Closing (≈5 seconds)

WHAT TO SAY

"Next steps: the full RQ1, RQ3 and RQ5 experiments, and fine-tuning the verifier. Thank you."

5-MINUTE TIME BUDGET

Slide 1: 0:10 · Slide 2: 0:30 · Slide 3: 0:15 · Slide 4: 0:40 · **Slide 5: 1:10** · Slide 6: 1:10 · Slide 7: 1:00 · Close: 0:05 → about 5:00.

PART F

Viva preparation

Likely questions with short answers, a glossary, and the numbers to remember.

20. Question bank

Question	Short answer
Isn't this agentic AI?	No. The pipeline is fixed and runs once. The LLM never decides what to do next, never chooses a tool and never loops. It only generates text, which we then verify.
Isn't splitting into claims like an agent breaking a task into subtasks?	No. It is an NLP operation on text (one sentence → several facts) at a fixed step. Agentic decomposition is an AI planning actions.
Isn't searching documents "tool use"?	No. Search is ordinary code that runs every time. In agentic AI, the model decides whether and when to call a tool; ours never decides.
Isn't the correction step an agent fixing its own work?	It runs exactly once, only on flagged claims. No loop and no decision to stop: it is one more generation step.
Is it just RAG?	RAG is the first half. Plain RAG stops after generation; we add claim extraction, NLI verification, self-consistency, scoring and conflict detection, evaluated on SciFact. It is "verified RAG".
What is NLI?	Natural Language Inference: given a premise and a hypothesis, decide entailment, contradiction or neutral.
Why NLI instead of asking the LLM whether it is right?	An LLM judging itself shares its own blind spots. DeBERTa is a separate model trained for exactly this task. We also built an LLM-judge baseline to compare against.
What is self-consistency?	We ask the same question 4 more times at temperature 0.8. Facts the model really knows reappear; hallucinated details change. (SelfCheckGPT, 2023.)
How is confidence computed?	0.6 × NLI entailment + 0.4 × self-consistency − 0.5 × contradiction, clipped to 0–1. Verified ≥ 0.70, unsupported < 0.40.
How is hallucination rate computed?	(Unsupported + contradicted claims) ÷ all claims, judged automatically by the NLI checker.

How do accuracy and hallucination rate relate?	Different things. The hallucination rate measures the LLM (judged by DeBERTa). Accuracy measures DeBERTa itself (judged by experts). Accuracy tells us how much to trust the hallucination rate.
Why is accuracy only 0.70?	3-way classification on hard scientific text; most errors are cautious; fine-tuning on SciFact is the next step.
Which models and why?	gpt-oss-120b: a strong open-weight LLM, free via Groq. DeBERTa-v3: one of the strongest open NLI models. MiniLM: small and fast embeddings. All are free and run on a laptop or free API.
What is SciFact?	A public benchmark (Wadden et al., 2020): 5,183 scientific abstracts and claims labelled by experts as supported, refuted, or not enough info.
What is an embedding?	A list of numbers (384 for MiniLM) representing a text's meaning; similar meanings have similar vectors.
What is temperature?	A setting for how random the LLM's word choices are; low = predictable, high = varied.
What does "grounded" mean?	The answer is based on the provided documents, not the model's memory, and cites them.
What are the limitations?	Contradiction detection (recall 0.59), hand-set weights, tables in PDFs, English only, free-tier rate limits.
Did you find any bugs?	Yes. NLI labels a passage that merely doesn't mention a fact as "contradiction"; our first scoring then flagged true claims. We fixed it by judging a claim by its best-supporting passage, and handling real disagreement separately in conflict detection.

21. Glossary

Term	Meaning
Hallucination	AI output that is false or not supported by any source.
LLM	Large Language Model: a huge decoder transformer that generates text (here gpt-oss-120b).
RAG	Retrieval-Augmented Generation: retrieve relevant passages and put them in the prompt before generating.
Chunk / passage	A ~220-word piece of a document.
Embedding / vector	Numbers representing meaning (384 per text for MiniLM).
Vector database	A database that stores vectors and finds the most similar ones (ChromaDB).
Cosine similarity	How closely two vectors point in the same direction (1 = same meaning).
Bi-encoder	Encodes texts separately into vectors (fast search).

Cross-encoder / re-ranker	Reads query and passage together to score relevance (accurate re-ranking).
Transformer / attention	The neural architecture behind modern AI; attention lets each word look at all others.
Encoder / decoder	An encoder understands text; a decoder generates text.
NLI	Natural Language Inference: entail / neutral / contradict.
Premise / hypothesis	In NLI: the evidence text / the claim being checked.
Claim (atomic)	A single, self-contained, checkable fact.
Self-consistency	Checking whether a fact reappears across several sampled answers.
Temperature	The randomness of the LLM's output.
Prompt engineering	Designing the instructions given to an LLM to get better output.
Zero-shot / few-shot	Instructions without examples / with a few worked examples.
Chain-of-thought (CoT)	Asking the model to reason step by step before answering.
Baseline	A simpler system you compare against.
Ablation	Removing one component to measure its contribution.
Benchmark	A standard public dataset for fair evaluation (SciFact).
Accuracy / precision / recall / F1	Overall correctness / correctness when predicting X / share of real X found / their balance.
Macro-F1	F1 averaged equally over all classes.
AUROC	How well a score separates two groups (0.5 = chance, 1 = perfect).
Confusion matrix	A table of true labels vs predicted labels.
Transfer learning	Reusing a model pre-trained on large data for a new task.
Agentic AI	AI that plans, chooses tools and acts in loops on its own. **Not** what VERITAS is.

22. Numbers to remember

What	Number
Demo corpus	7 papers on LLM hallucination
SciFact	5,183 abstracts · 300 dev claims · 340 claim–abstract pairs
RQ2 accuracy / macro-F1 / AUROC	**0.70 / 0.69 / 0.86**
Contradiction recall (weakest point)	**0.59**

"Supported" precision	0.75 (right about 3 times out of 4)
RQ4: bad prompt vs our prompt	**74% vs 0%** hallucination (29/39 vs 0/15 claims)
Confidence formula	0.6 × NLI + 0.4 × consistency − 0.5 × contradiction
Thresholds	verified ≥ 0.70 · unsupported < 0.40
Retrieval	top 20 → re-rank → top 5, at most 2 per document
Temperatures	0.1 (answer) · 0.8 (samples) · 4 samples
Chunk size	~220 words, 40-word overlap
Embedding size	384 numbers (MiniLM)
GPU speed-up	6.4× (0.26 s vs 1.67 s per check)
Time per question	Quick ≈ 10–40 s · Deep ≈ 20 s – 2 min